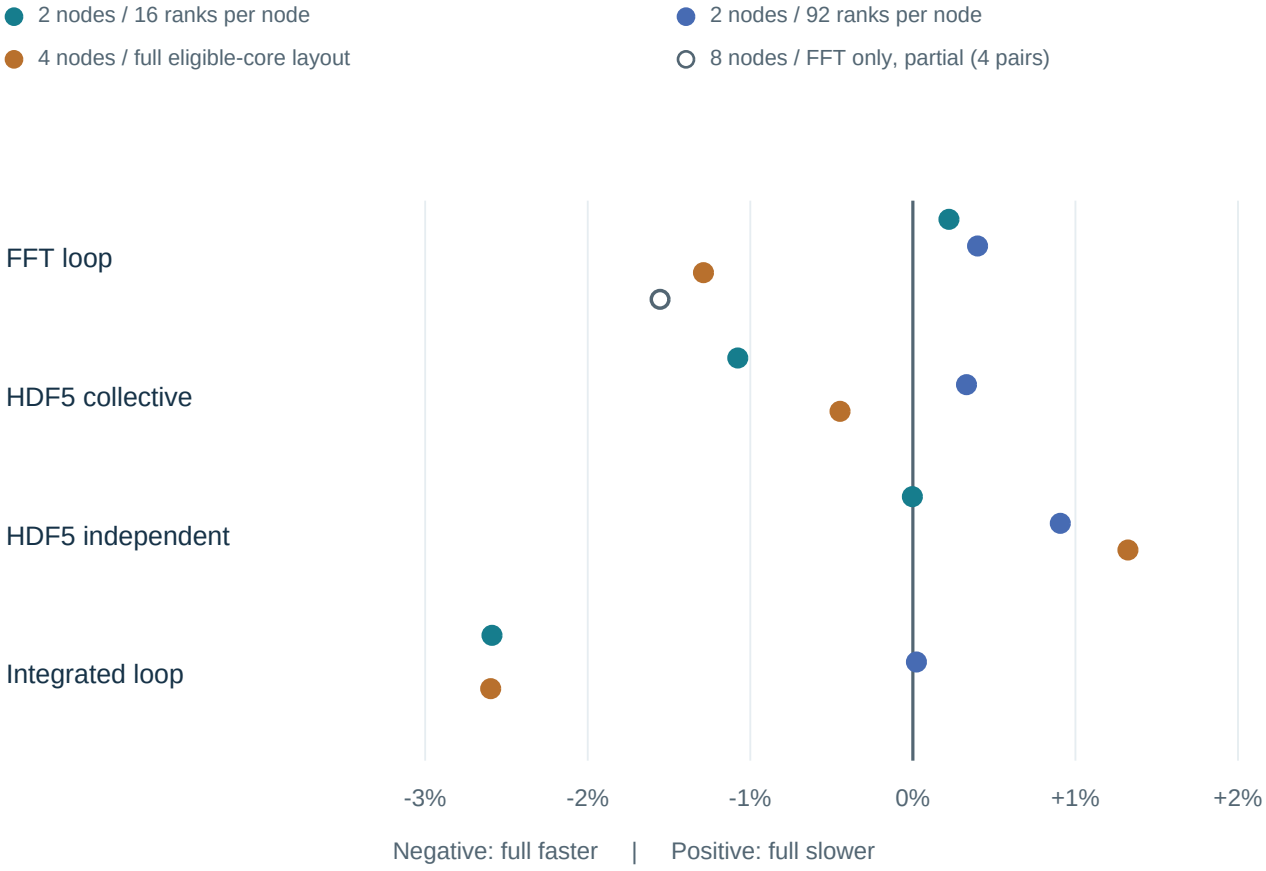
Encouraging results; confirmation pending

No consistent performance penalty is apparent in the completed tested conditions. Small differences in both directions do not establish zero overhead or a hardening speedup. Testing is paused; this report records the evidence available at the end of the session.

The dense campaign compared a hardened caller and its complete trial-owned library/dependency stack with matching reference builds. Completed conditions cover 2-node sparse and dense layouts and a 4-node dense layout. The 8-node FFT condition stopped after four of ten requested pairs when its budget expired.

Dense MPI workloads: observed runtime change



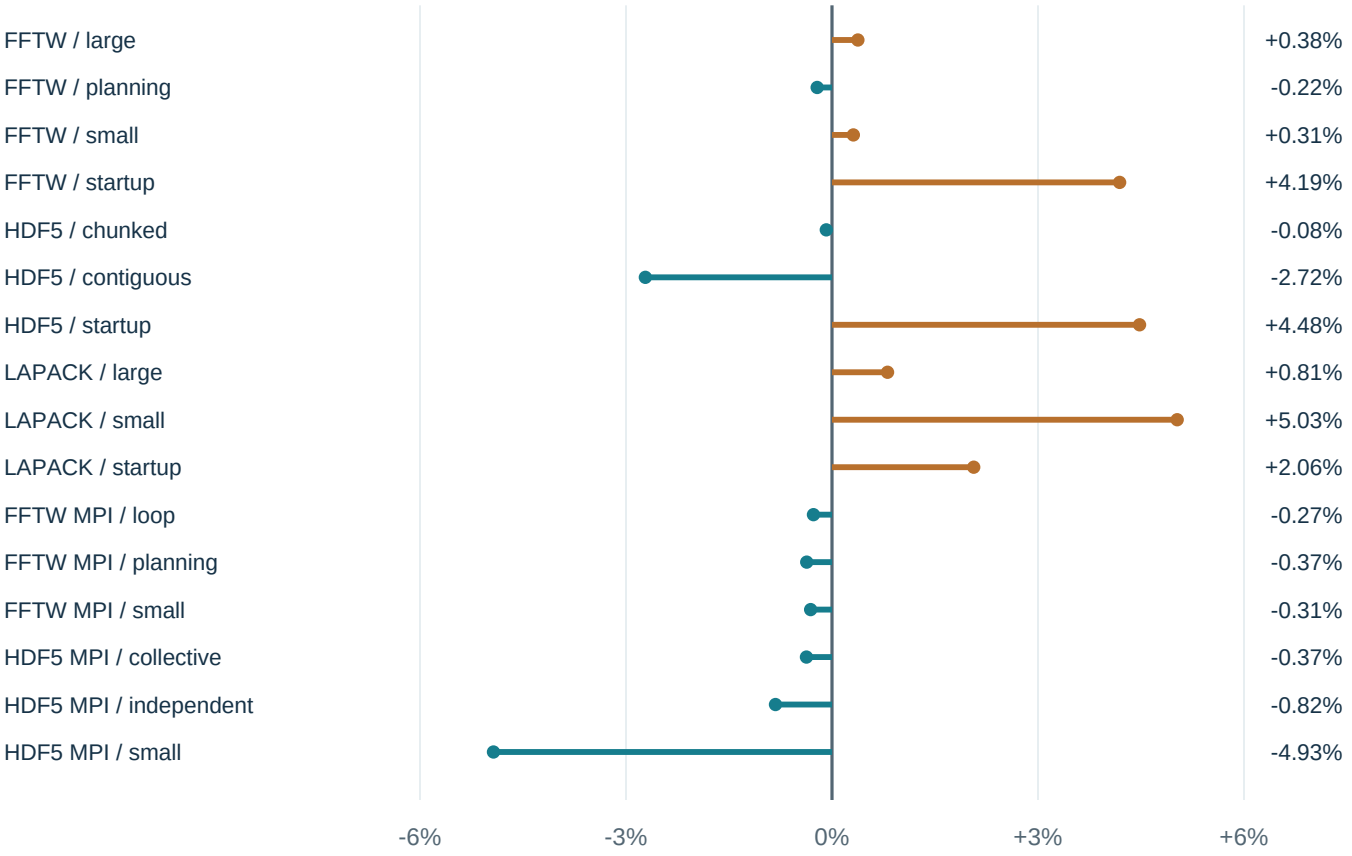
Read this plot as point estimates, not a significance test. Each completed condition has 10 pairs. Confidence bounds are omitted because the final photos do not expose all bounds reliably; no error bars have been invented. Layouts may differ in calibrated work, so these are not strong-scaling curves.

Across the 12 completed dense conditions, point estimates range from about **-2.60% to +1.32%**. The incomplete 8-node FFT estimate is about **-1.56%**. No completed paired 8-node HDF5 or integrated-loop result was shown.

Package pilot and completed coverage

The latest all-package pilot screenshot reports complete coverage of 16 workloads, with ten pairs per workload. The chart records its displayed point estimates. Startup and small-case variation can be large relative to the measured work.

Latest screening pilot: observed runtime change



Negative means full was faster; positive means full was slower. These pilot point estimates have substantial uncertainty: for example, the latest FFTW MPI loop is **-0.270% [-1.541%, +0.929%]**, and the MPI small HDF5 case is about **-4.93% [-30.90%, +29.44%]**, using the displayed intervals. Both span zero. Earlier pilot estimates also changed between runs.

What was accomplished. The operator reported full/reference package build completion; local and inter-node MPI qualification reached finalization for both arms. The package screening pilot completed, followed by denser FFT, collective/independent HDF5, and integrated-loop comparisons. The runner now includes CPU-grant discovery and WEKA reservation checks.

Coverage is not acceptance. A passing run records successful execution under the runner checks. It does not prove an overhead threshold is met, validate every user application, or measure the security benefit. Original build, binding, fabric and pair-level records still need review.

What this means for supplied libraries

The deployment question remains useful. Users can call a hardened shared library while building their application with ordinary compiler options, subject to the normal ABI/toolchain requirements. Library compiler options do not automatically apply to the application source. Checks such as fortified libc call sites depend on the calling source compilation [1, 2].

The current experiment changes both the caller and its matching stack. A small combined difference cannot isolate the library contribution: caller and library effects can reinforce or offset each other. A real application may exercise different library paths, problem sizes, data movement, and checkpoint frequencies.

The next comparison should match that deployment

Use **one fixed, ordinarily compiled caller executable** with the reference library/dependency stack and then the hardened stack. Verify the intended libraries on every rank; retain compatible headers and interfaces. Keep input, work, placement and runtime policy fixed within each pair. This comparison is planned and is **not yet implemented** in the production runner.

Whole-job impact depends on time spent in affected paths. If a library occupied 40% of a fixed-work job and became 1% slower, the simplest unchanged-behavior estimate would be about 0.4% longer job time. This is an illustration, not a result from today. Repetition accumulates absolute seconds; the percentage does not multiply by iterations or node count. Communication and I/O can change this relationship.

Resume next week with the existing evidence

1. Preserve original result folders and inspect complete paired timings, build identities, effective flags, loaded libraries, CPU bindings and HDF5 modes. Do not discard slow points simply because they change the result.
2. Recompute paired intervals from the raw data; keep allocations and work definitions separate. Agree on a practically meaningful overhead threshold and the precision needed to assess it.
3. Complete the missing 8-node conditions with recorded, frozen work and repeat selected cases in an independent allocation. Then add the fixed caller comparison and one representative application. Individual-flag experiments remain deferred.

Evidence and method. Figures transcribe operator screenshots from 16 October; no original HPC result archive was available for this report. The production implementation reports 100 x (mean full / mean reference - 1) over complete pairs and resamples paired process runs for a 95% percentile bootstrap interval (2,000 resamples). Ranks and loop iterations are not independent experimental replicates. The charts preserve each layout and the partial-result status; no pooled overall effect is claimed.

[1] GCC 12.5, Instrumentation Options: official compiler documentation.
[2] GNU C Library, Source Fortification: official library documentation.